1 La boucle for

La syntaxe est :

```
for compteur=debut:{pas:}fin
    :
end
```

Les constantes réelles `debut` et `fin` sont les bornes inférieures et supérieures de l'intervalle à parcourir. La constante positive ou négative `pas` est l'incrément qui par défaut vaut 0.

```
>> for i=1:2:10 i*i, end
ans  =
1.

ans  =
9.

ans  =
25.

ans  =
49.

ans  =
81.
```

On peut remplacer `compteur=debut:pas:fin` par `compteur=u` où `u` désigne un vecteur, la variable `compteur` décrit alors toutes les composantes de u.

```
>> u=[1 5 7 9];

>> for compteur=u compteur/2, end
ans  =
0.5
ans  =
2.5
ans  =
3.5
ans  =
4.5
```

Dans le cas où `u` est une matrice, l'instruction `for compteur = u` signifie "pour compteur décrivant les colonnes de u".

```
>> u=[1 2 ; 3 4 ];
>> for compteur = u, norm(compteur), end
ans  =
3.1623

ans  =
4.4721
```

On peut utiliser la boucle `for` pour définir une matrice :

```
>> n=4; a=zeros(n,n);

>> for i=1:n-1, a(i,i)=2; a(i,i+1)=-1; a(i+1,i)=-1; end;

>> a
a  =
2    -1     0     0
-1     2    -1     0
0    -1     2    -1
0     0    -1     0
```

On peut sortir d'une boucle `for` par la commande `break`. Cette commande fonctionne uniquement avec `for` ou `while`. dans d'autres cas, on utilise `return`.
Les boucles `for` peuvent être imbriquées.

```
>> s = 10;
H = zeros(s);

for c = 1:s
for r = 1:s
H(r,c) = 1/(r+c-1);
end
end
```

Crée la matrice de Hilbert d'ordre 10.

2 La boucle while

La syntaxe est :

```
while expression_logique
    :
end
```

Les instructions sont exécutées tant que le booléen `expression_logique` est vrai.

```
>> x=1;
>> while x-exp(-x) > eps  x=x-0.001; end;

>> x
x  =
0.567
```

On peut sortir d'une boucle `while` par la commande `break`.

3 L'alternative if

La syntaxe est :

```
if condition
  instructions1
      else
  instructions2
      end
```

La partie `else` peut être facultative.
`condition` est une expression booléenne.
Quand `else` est suivi d'un `if` on peut utiliser la forme :

```
if condition1
  instructions1
elseif condition2
  instructions2
      else
  instructions3
      end
```

```
>> [m,n]=size(A); [p,q]=size(B);
>> if (n==p)
 C=A*B;
 elsif (q==m)
 C=B*A;
 else
 error('produit impossible')
 end
```

4 *Instruction switch case*

Avec l'instruction `switch case` l'exécution des différents choix est basée sur la valeur d'une variable ou d'une expression et pas sur des expressions logiques comme dans `if`.
La syntaxe est :

```
switch selecteur
case value1
instructions 1
case value2
instructions 2
...
case valuen
instructions n
[otherwise instructions]
end
```

Si `selecteur` est égale à `value1` les instructions correspondantes `instructions 1` sont exécutées...Dans la variante avec `otherwise`, si `selecteur` n'est égale à aucune des valeurs `value1`,...,`valuen`, les instructions `instructions` sont exécutées.
La différence avec l'alternative `if`, est que l'exécution des différents choix est basée sur la valeur d'une variable et non sur une expression logique.

```
>> n = input('Enter a number: ');

switch n
case -1
disp('negative one')
case 0
disp('zero')
case 1
disp('positive one')
otherwise
disp('other value')
end
```

5 Commandes tic toc

Les fonctions `tic` et `toc` renvoient le temps écoulé. Ceci permet de comparer les performances des algorithmes.
La précision est de 1/10 de seconde.

```
>> tic
A = rand(12000,4400);
B = rand(12000,4400);
toc
Elapsed time is 1.118728 seconds.
```

Comme nous l'avons déjà dit, MATLAB est conçu pour le calcul matriciel. Les opérations performantes sont celles qui comportent des opérations matricielles ou vectorielles. En voici une illustration :

```
>> clear

>> tic;i=0; for t=0:0.01:100, i=i+1; y(i)=sin(t); end; toc
Elapsed time is 0.109942 seconds.

>> tic; t=0:0.01:100; y=sin(t); toc
Elapsed time is 0.104575 seconds.
```

La vectorisation consiste à transformer des boucles en opérations matricielles ou vectorielles équivalentes.

6 Quelques fonctions internes

6.1 Fonctions logiques

Voici une liste qui n'est pas exhaustive de fonctions logiques :

- `&` : et logique
- `&&` : et logique version scalaire
- `|` : ou logique
- `||` : ou logique version scalaire

- ~ : négation

La version scalaire du "et" , "ou" logique permet une évaluation rapide en anticipant le résultat. Par exemple pour l'opération logique suivante : `(x>5)&&(y>5)`
si $x < 5$ alors le résultat est faux, quelque soit la valeur de y.

6.2 La fonction find

La commande `find(a>c)` renvoie les indices des éléments de la matrice a plus grands que c.
Elle permet de remplacer une boucle.

```
>> A=rand(1,5)
A  =
0.6292    0.2467    0.1881    0.8661    0.9904
>> w=find(A<0.4)
w  =
2     3
```

6.3 La fonction de tri

La fonction `sort` trie les composantes d'un vecteur. Le tri est effectué par ordre croissant.
Si le vecteur est complexe, le tri est fait sur le module des composantes.
Si c'est une matrice, le tri est fait colonne par colonne de manière indépendante.

```
>> a=[6 -1 2]
a  =

6.  -1.   2.

>> sort(a)
ans  =

 -1     2     6

>> H=hilb(4)

H =

1.0000    0.5000    0.3333    0.2500
0.5000    0.3333    0.2500    0.2000
0.3333    0.2500    0.2000    0.1667
0.2500    0.2000    0.1667    0.1429

>> sort(H)

ans =

0.2500    0.2000    0.1667    0.1429
0.3333    0.2500    0.2000    0.1667
0.5000    0.3333    0.2500    0.2000
1.0000    0.5000    0.3333    0.2500
```

On peut mettre une deuxième argument pour récupérer les indices triés.

```
>> x=randperm(6) % effectue une permutation aléatoire des entiers de 1 a 6
x =
1     6     2     5     3     4

>> [xt,it]=sort(x)
xt =
1     2     3     4     5     6

it =
1     3     5     6     4     2
```

6.4 Les fonctions min et max

Les fonctions `min` et `max` permettent de trouver le plus grand ou plus petit élément d'un vecteur (et en option on peut aussi récupérer l'indice qui réalise le minimum ou le maximum).

```
>> u = [1,2,3,4,5,6]
u =
1     2     3     4     5     6

>> max(u)
ans =
6

>> [m,it]=min(u)
m =
1

it =
1
```

Lorsque l'argument d'entrée est une matrice, la recherche du minimum ou maximum se fait colonne par colonne. Les fonctions retournent alors un vecteur ligne des min ou max sur chaque colonne.

```
>> B = [7,8,-1;2,3,4]
B  =
7     8    -1
2     3     4

>> C = max(B)
C  =
7     8     4
```

Pour avoir l'élément max ou min d'une matrice A, il suffit de faire la recherche sur le vecteur correspondant `A(:)`.

```
>> A
A =
1     2     3
4     5     6
```

```
>> min(A(:))
ans =
1
```

6.5 Les fonctions sum et prod

Les fonctions `sum` et `prod` permettent de faire la somme ou le produit des éléments d'un vecteur. Les règles pour une matrice sont les mêmes que pour les fonctions `min` et `max`.

```
>> u
u =
1     2     3     4     5     6

>> sum(u)

ans =
21

>> A = [1,2,3;4,5,6]
A  =
1.   2.   3.
4.   5.   6.

>> sum(A)

ans =
5     7     9 % somme sur les colonnes de A

>> sum(A,2) % somme sur les lignes de A

ans =
6
15

>> prod(A)
ans =
4    10    18 % produit sur les colonnes de A
>> prod(A,2) % produit sur les lignes de A
ans =
6
120
```